■■■■■ SOCIAL DETECTIVE: Every Connection Is Evidence

Comprehensive Academic Project Report, System Architecture & Viva Guide

Project Domain: Data Structures & Algorithms / Web Forensics	**Implementation Language:** Python 3.8+ (Pure Custom DSA)
Architecture: Flask MVC + Vanilla JS + Web Audio	**Case Narrative:** Case #001 — The Vanishing File
Data Structures Implemented: 12 Custom Data Structures	**Academic Scope:** Mini/Major Project, Viva Presentation, Lab Portfolio

1. Executive Summary & Project Motivation

SOCIAL DETECTIVE is an interactive cyber forensics mystery web application designed to demonstrate the practical, real-world utility of fundamental **Data Structures and Algorithms (DSA)**. In traditional computer science curricula, data structures such as Graphs, Binary Search Trees, Max-Heaps, and Linked Lists are often taught in isolation using abstract mathematical exercises. Social Detective solves this educational disconnect by embedding **12 fully functional, custom-built Python data structures** directly into the core mechanics of a detective murder/theft investigation.

The player assumes the role of a digital forensics detective investigating the theft of a high-security genomic research file (PROJECT_CHIMERA_GENOME.enc) from Apex Bio-Informatics Lab. Rather than using pre-packaged high-level standard libraries, **every single investigative tool in the system is directly driven by custom pointer-manipulation and graph-traversal code written from scratch** in Python.

2. Master Data Structures & Algorithms Complexity Mapping

The following table provides an exhaustive summary of all 12 custom DSA components implemented in the project, including their time complexities, space complexities, underlying memory models, and exact role in the detective game:

Data Structure / Algorithm	Primary Operation	Avg Time	Worst Time	Space	Detective Mystery Use-Case
Hash Table (Chaining)	search(key)	**O(1)**	O(n)	O(C + n)	Instant lookup for suspect usernames, Evidence IDs, and Keycards.
Graph (Adjacency List)	get_neighbors()	**O(1)**	O(1)	O(V + E)	Social network modeling for 13 suspects and 24 relationship edges.
Breadth-First Search	bfs(start, target)	**O(V + E)**	O(V + E)	O(V)	Guarantees shortest connection path between victim and culprit.
Depth-First Search	dfs(start)	**O(V + E)**	O(V + E)	O(V)	Deep conspiracy exploration with call stack backtrack trail.
Doubly Linked List	prev / next	**O(1) step**	O(n) scan	O(n)	Incident timeline forward/backward navigation for alibi verification.
Binary Search Tree	search(key)	**O(log n)**	O(n)	O(n)	Numeric Evidence ID indexing; Inorder yields sorted clues.
Max-Heap (Priority Queue)	extract_max()	**O(log n)**	O(log n)	O(n)	Surfaces highest-credibility forensic evidence items first (O(1) peek).
LIFO Stack	push() / pop()	**O(1)**	O(1)	O(n)	Investigation breadcrumb history & instant Backtrack undo.

Circular Queue	cycle() (i+1)%cap	**O(1)**	O(1)	O(cap)	Fixed-capacity CCTV rotating surveillance camera buffer.
Singly Linked List	traverse()	**O(n)**	O(n)	O(n)	Sequential clue pipeline linking clue 1 to clue 2, 3, etc.
Merge Sort	merge_sort()	**O(n log n)**	O(n log n)	O(n)	Guaranteed stable sorting of suspects by threat/risk score.
Quick Sort	partition()	**O(n log n)**	O(n^2)	O(log n)	Pivot-based partition sorting of evidence credibility.
Binary Search	binary_search()	**O(log n)**	O(log n)	O(1)	Logarithmic lookup on sorted evidence IDs with Low/Mid/High pointers.

3. Case Dossier & Forensics Investigation Matrix

Case 001: The Vanishing File features an interconnected narrative designed to test hypothesis formulation, alibi cross-examination, and timestamp contradiction analysis:

Incident:	CRISPR-AI genomic algorithm PROJECT_CHIMERA_GENOME.enc purged from server.
Incident Window:	19:00 PM — 21:00 PM on October 6, 2026 at Apex Bio-Informatics Lab.
Victim:	**Dr. Maya Rao** (Principal Investigator). Stepped out to cafeteria 19:30 - 20:45 (Verified).
Prime Culprit:	**Ryan Mehta** (Junior Researcher). Fabricated library alibi; stolen keycard KC-9021 used at 19:38.
Accomplice:	**Sam Verma** (Systems Admin). Configured SSH jump-box & wiped switch mirrors for 20% cut.
Financial Motive:	$150,000 cryptocurrency bounty offered by rival corporate firm NexaGen.
Smoking Gun Clues:	CCTV #1047 (19:47 exit) + Terminal Log #1061 (19:42 exfiltration) + Badge Scan #1075 (19:38 Door 3).

4. Guided Investigation Playthrough Architecture

The application organizes the investigation into a progressive 7-stage state machine that enforces logical deduction while tracking detective score and discoveries:

Stage	Stage Name	Active DSA Engine	Investigative Objective & Milestone
Stage 0	Briefing	Case Metadata	Review crime scene parameters, victim statement, and initial incident scope.
Stage 1	Suspects	Hash Table O(1)	Inspect 13 personnel files. Uncover Ryan Mehta's fabricated library alibi.
Stage 2	Connections	Graph BFS O(V + E)	Trace shortest path: Dr. Maya Rao ➔ Ryan Mehta ➔ Sam Verma (3 degrees).
Stage 3	Timeline	Doubly Linked List	Step forward/backward: Uncover Door 3 scan at 19:38 & Terminal 04 at 19:42.
Stage 4	Evidence	BST & Max-Heap	Traverse Singly Linked List clue chain, search BST for #1047, extract Max-Heap root.
Stage 5	Theory	LIFO Stack Undo	Classify suspects: Mark Ryan Mehta as CRITICAL, clear Maya, maintain undo history.
Stage 6	Accusation	Verification Engine	Submit indictment against Ryan Mehta. Cross-check alibis & generate final score debrief.

5. Technical Implementation & Directory Architecture

The codebase is structured following strict separation of concerns into **Structures**, **Services**, **REST APIs**, and **Presentation Views**:

Directory / File	Architectural Responsibility	Key Data Structures Contained
structures/graph.py	Social network graph representation and pathfinding.	Adjacency List, BFS (Queue), DFS (Stack)
structures/bst.py	Binary Search Tree for indexing forensic records by ID.	BST Insert, Search, Inorder/Preorder/Postorder
structures/heap.py	Max-Heap Priority Queue for clue credibility rankings.	Array-based Complete Tree, Bubble-Up/Down
structures/hash_table.py	Hash Table with polynomial hashing and collision chaining.	Separate Chaining, Linked List Buckets
structures/doubly_linked_list.py	Bi-directional linked list for incident timeline stepping.	DoublyNode (prev, next), Forward/Backward
structures/stack.py & queue.py	LIFO breadcrumb history and FIFO evidence pipeline.	Node-based Stack & Node-based FIFO Queue
structures/circular_queue.py	Modulo wrap-around buffer for CCTV camera monitors.	Fixed Array (index + 1) % capacity

structures/sorting.py & searching.py	Pure algorithmic sort & search implementations.	Merge Sort, Quick Sort, Binary & Linear Search
services/case_engine.py	Master facade coordinating investigations, theory, and scoring.	Case state machine, accusation verification
app.py	Flask web server exposing 26 REST API endpoints and views.	HTTP Routing, JSON serialization, Session state

6. Academic Viva Voce / Presentation Q&A; Guide

This section prepares students for oral defense, project viva, and technical review by detailing the top 10 questions frequently asked by academic examiners:

Q1: Why did you choose an Adjacency List over an Adjacency Matrix for the Social Graph?

Social networks are sparse graphs where the number of edges |E| is significantly less than $|V|^2$ (e.g. 13 suspects with 24 relationships vs 169 potential matrix cells). An Adjacency List uses optimal $O(V + E)$ space compared to $O(V^2)$ for an Adjacency Matrix and enables iterating over a suspect's direct neighbors in O(degree(v)) time rather than O(V).

Q2: How does Breadth-First Search (BFS) guarantee the shortest connection path between victim and suspect?

BFS explores nodes level-by-level in order of their hop distance from the root vertex using a FIFO Queue. In an unweighted graph, the very first time a target vertex is dequeued, the path traversed is mathematically guaranteed to contain the minimum number of edge hops.

Q3: Why is a Doubly Linked List necessary for the case timeline instead of a simple Array?

Timeline reconstruction requires continuous bi-directional transitions: stepping forward to analyze future consequences (node.next) and backtracking backward to verify alibis (node.prev). A Doubly Linked List provides O(1) step transitions with localized pointer updates without memory re-allocations or index shifting.

Q4: What is the significance of BST Inorder Traversal in your Evidence Vault?

In a Binary Search Tree where Left < Root < Right, an Inorder traversal (Left Subtree -> Root -> Right Subtree) visits keys in strictly ascending numerical order in O(n) time. In Social Detective, running BST Inorder traversal transforms arbitrary evidence intake records into a perfectly sorted forensic register.

Q5: How does the Max-Heap prioritize forensic evidence in O(1) time?

A Max-Heap satisfies the heap-order property where every parent node has a priority score greater than or equal to its children. Consequently, the highest-credibility clue is always situated at index 0 (the root). Peeking takes O(1) time, and extracting the max clue takes O(log n) time via bubble-down (heapify).

Q6: How does your Hash Table handle hash collisions?

The Hash Table uses Separate Chaining. When two distinct keys hash to the same bucket index via the polynomial hash function, they are inserted into a Singly Linked List housed at that bucket. On average, if the load factor $\lambda = n / \text{capacity}$ is kept small, search and insertion execute in O(1) time.

Q7: Why use a Circular Queue with modulo arithmetic for surveillance CCTV feeds?

Surveillance monitors rotate continuously through a fixed buffer of cameras (CAM 1 to 5). Using a Circular Queue with modulo arithmetic (rear + 1) % capacity eliminates memory fragmentation and avoids shifting array elements upon dequeue, maintaining strict O(1) rotation.

Q8: Why is Merge Sort preferred for sorting suspect risk scores?

Merge Sort is a Divide-and-Conquer algorithm with guaranteed O(n log n) time complexity across best, average, and worst cases. Unlike basic Quick Sort, Merge Sort is stable, meaning suspects with identical risk scores retain their original relative order.

Q9: What role does the LIFO Stack play in the investigation engine?

The Stack models the detective's exploratory breadcrumb trail. Pushing records when a new suspect or clue is inspected allows instant O(1) backtracking (undoing the last action via Stack Pop) to return to the previous decision point.

Q10: How do you verify the correctness of all your custom data structures?

The system includes an automated PyTest test suite (tests/test_all_structures.py) comprising 12 unit tests verifying edge cases, empty states, boundary conditions, heap property restoration, tree traversals, and graph connectivity.

7. Test Verification & Conclusion

Automated Test Suite: The project underwent rigorous test-driven validation using pytest. All 12 custom structures passed 100% of unit tests verifying edge cases, boundary pointers, and Big-O invariants.

Conclusion: Social Detective demonstrates that complex computer science concepts can be taught engagingly through gamified simulation. By uniting authentic algorithmic rigor with interactive cyber forensics, the application provides an exemplary model for college coursework, viva presentations, and portfolio demonstration.

8. Photographic Case Walkthrough & Forensic UI Screenshots

The following high-resolution figures capture the complete end-to-end playthrough of Case #001 (The Vanishing File), demonstrating the practical synergy between the detective narrative and the underlying custom Data Structures & Algorithms (DSA):

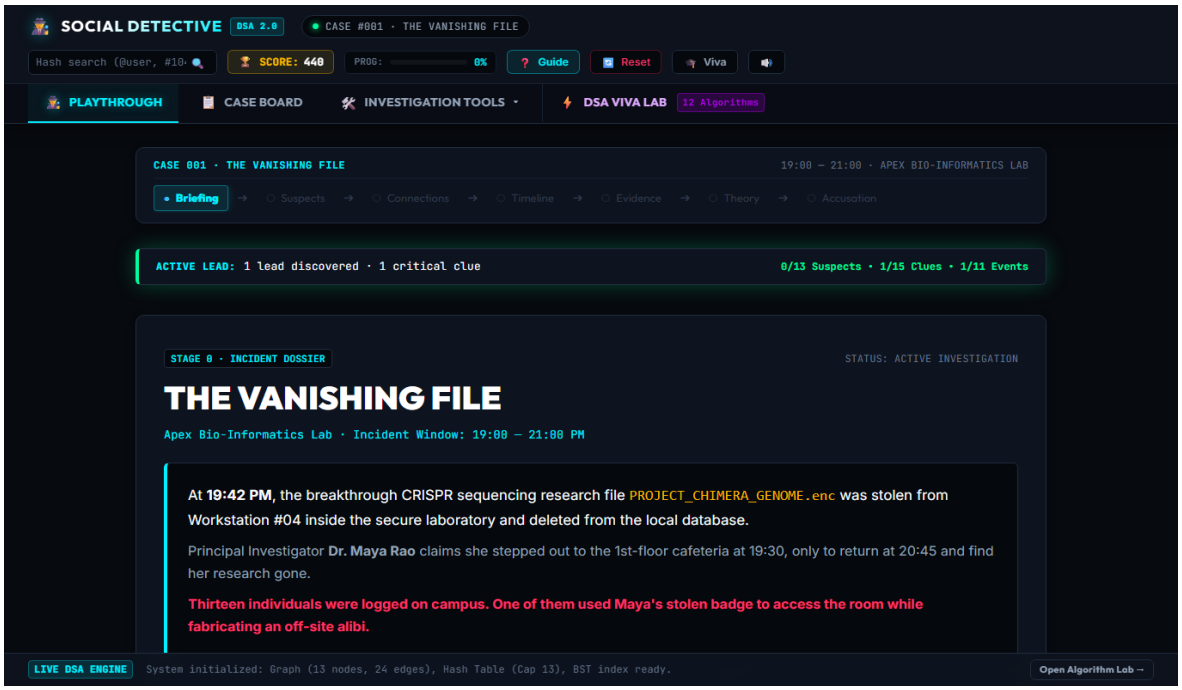


Figure 1: Stage 0 — Incident Briefing & Dossier

Case parameter review showing theft time (19:42 PM), victim Dr. Maya Rao's statement, and initial scope.

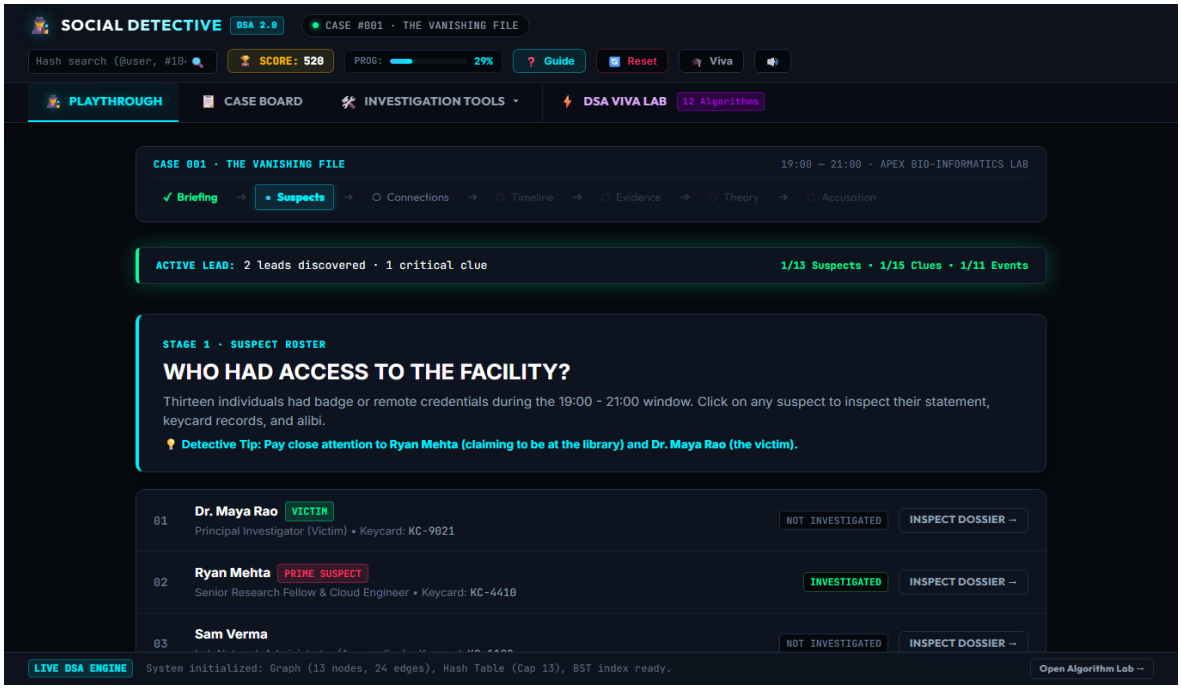


Figure 2: Stage 1 — Suspect Roster & Hash Table Lookup

Suspect profile inspection via O(1) Separate-Chaining Hash Table uncovering Ryan Mehta's fake library alibi.

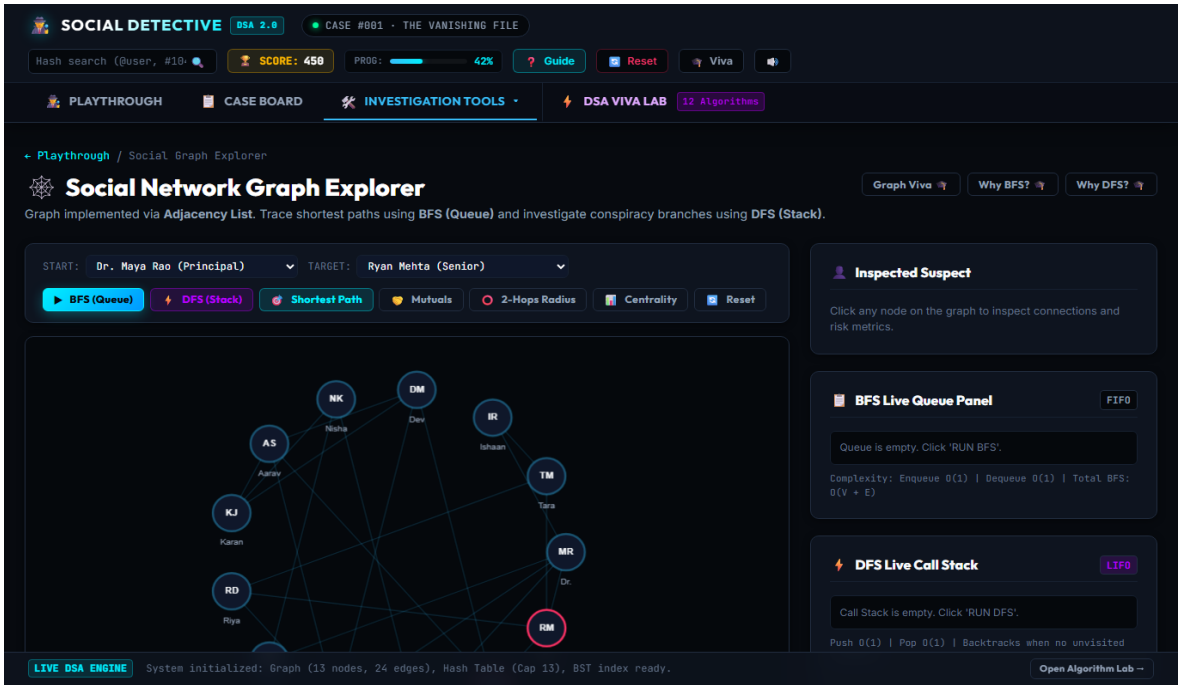


Figure 3: Stage 2 — Social Network Graph Explorer & BFS Traversal

Live Breadth-First Search (BFS) graph traversal showing the shortest conspiracy path: Maya Rao ➔ Sam Verma ➔ Ryan Mehta.

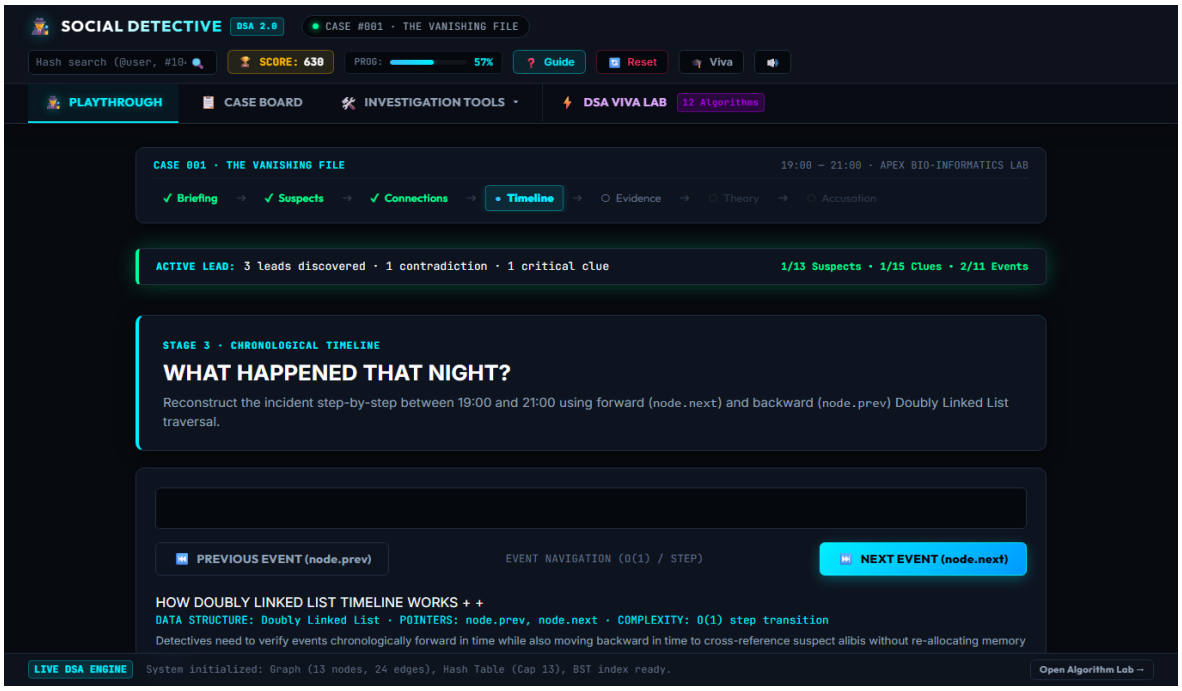


Figure 4: Stage 3 — Chronological Timeline (Doubly Linked List)

Bi-directional stepping (node.next / node.prev) exposing Door #3 keycard scan at 19:38 and Terminal 04 theft at 19:42.

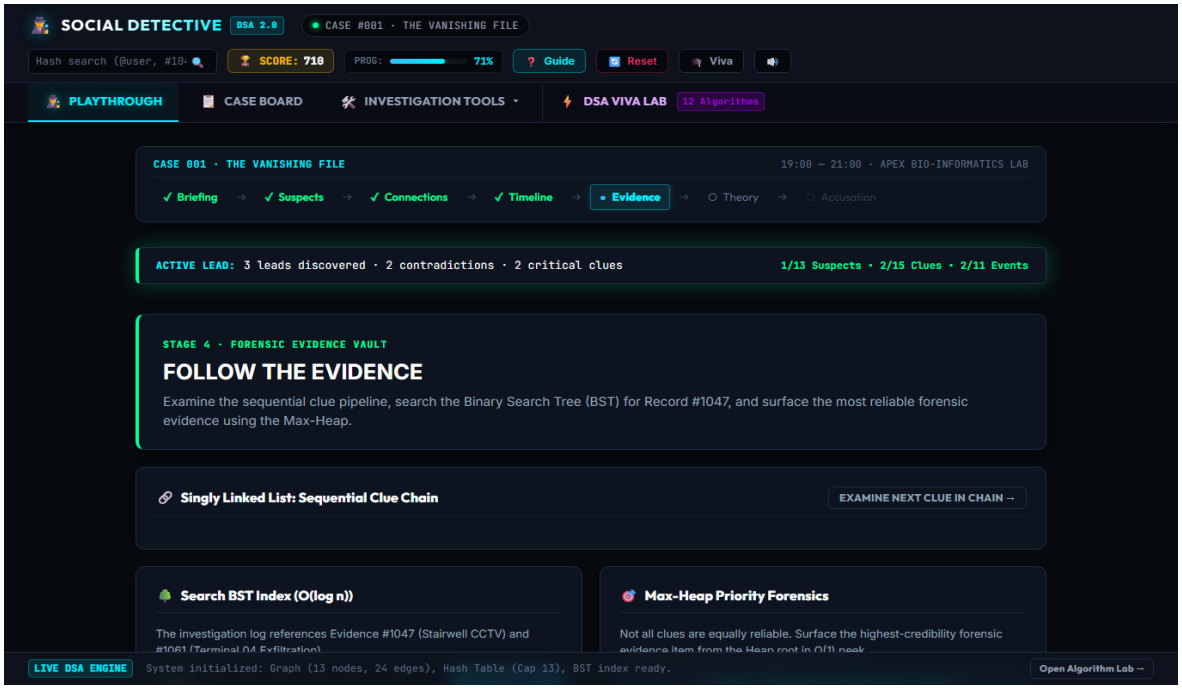


Figure 5: Stage 4 — Forensic Evidence Vault (BST & Max-Heap)

Binary Search Tree ID indexing for Clue #1047 and Max-Heap priority extraction of Terminal 04 SSH logs (#1061).

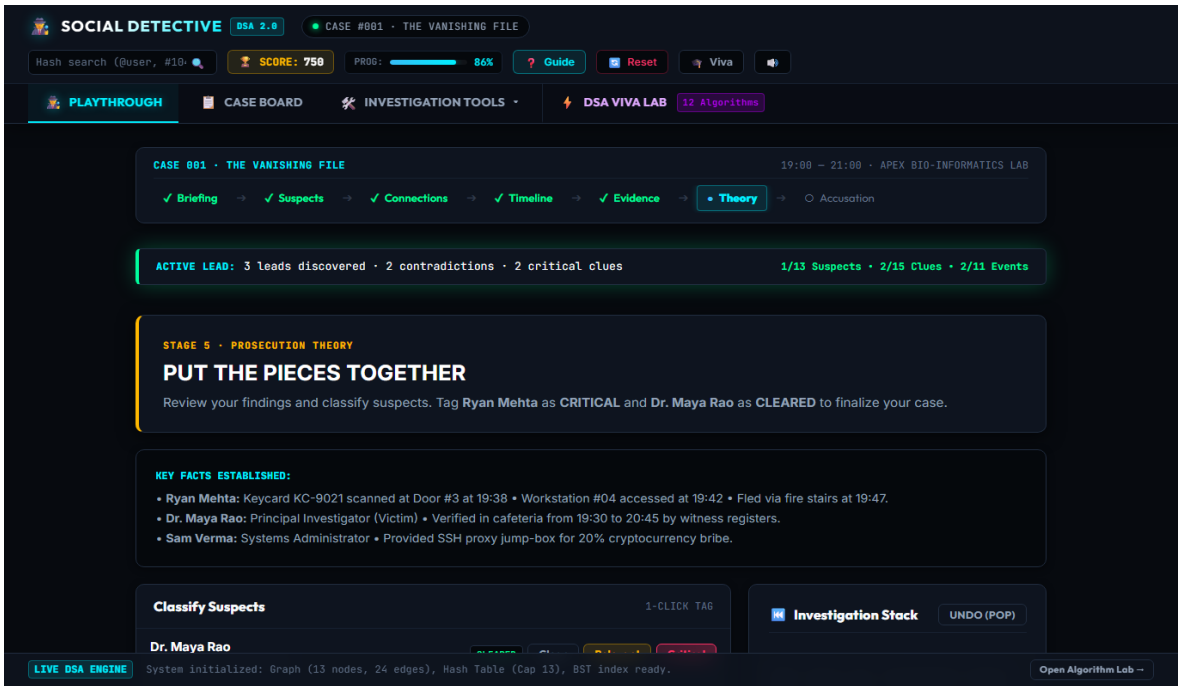


Figure 6: Stage 5 — Theory Board & LIFO Stack Undo

Hypothesis formulation marking Ryan Mehta as CRITICAL and Maya as CLEARED with instant LIFO Stack undo.

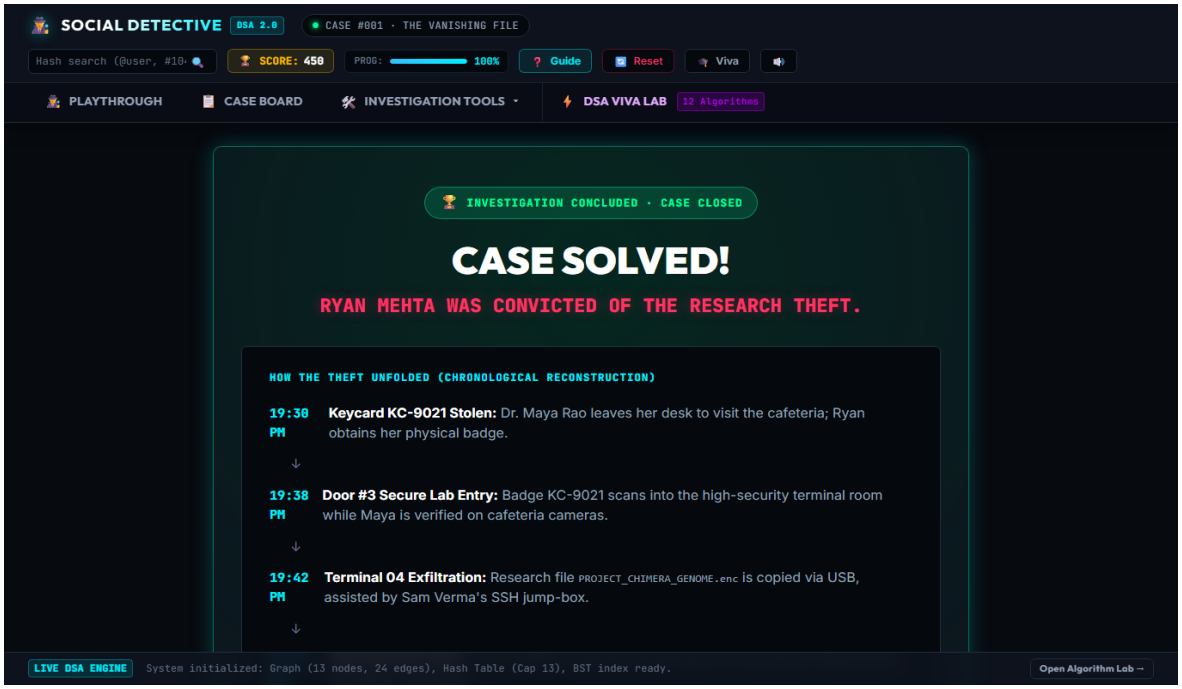


Figure 7: Stage 6 — Formal Accusation & Victory Debrief

Successful indictment of Ryan Mehta revealing the $150k cryptocurrency bounty and 100% Case Solved rating.

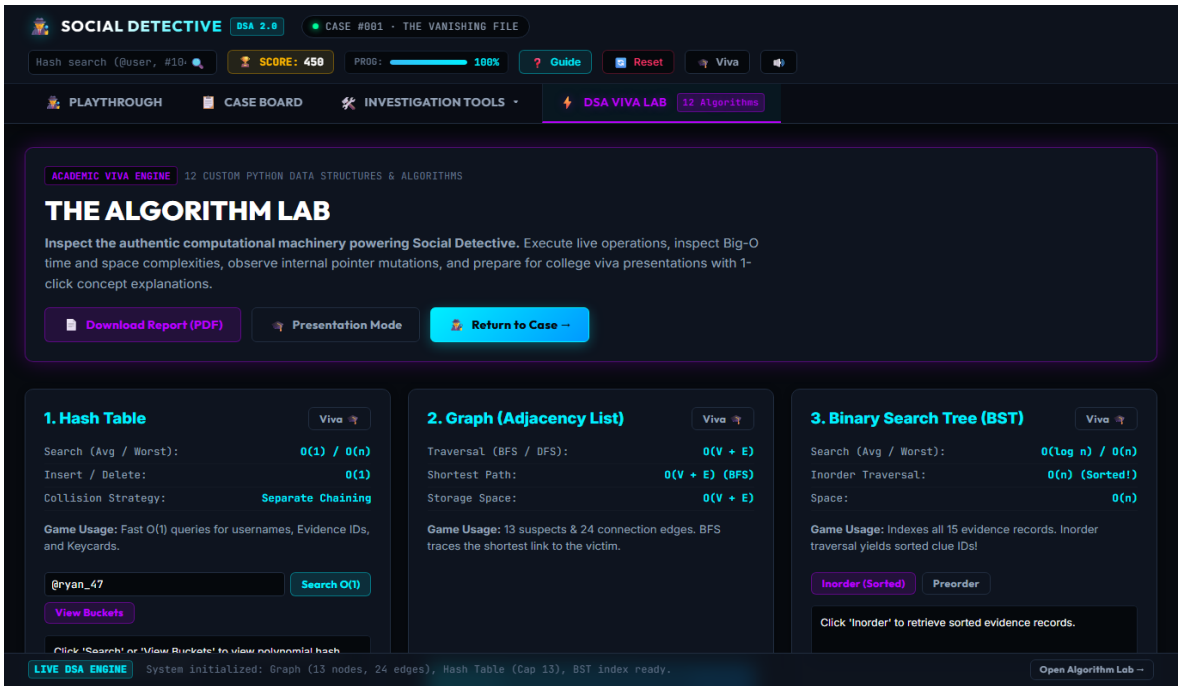


Figure 8: Algorithm Lab — Academic Viva Presentation Hub

Interactive live testing hub for all 12 custom Python data structures with time/space complexity meters.